Video Game Project

Hubert Malon

H00491052

HM4012@hw.ac.uk

EECE (Year 1)

31/03/2026

Video Game Project

Introduction

Early on, brainstorming went through a few concepts, with things such as a basic buggy building game or something along the lines of a rhythm game like Guitar Hero – but upon realising that synchronising everything to music and having a scrolling map to design and implement would simply be too complex, a basic maze runner game was settled on.

Theory

My concept phase began with a very basic thought of how to generate a maze. With the initial concept being a scrolling game that had an entry at the bottom and an exit at the top – with the intention of travelling between „sections" of the maze infinitely. I was planning on achieving this with some sort of random walk that generates the initial path between the start and finish points, with smaller random walks that branch out every few steps that the walk makes. I later realised that this would end up with quite plain looking mazes that were easy to decode, so I decided to do some research on maze generation algorithms.

When researching, I realised that maze generation was less about cells and walls and more about nodes with directions, which piqued my interest in the topic and got me excited about what method of bringing this into a practical game. A term I started coming across very early on was the term „perfect maze" which refers to a maze in which any two points in the maze are connected by no more than one path, meaning the maze is devoid of any inaccessible isolated areas, or any loops that branch out and connect back in. I immediately opted to find some sort of algorithm that enabled this, as that would naturally make for a more interesting game.

After looking through a wide range of algorithms, the one I settled on was Origin Shift. I chose this because it guarantees a perfect maze regardless of how long the maze cycle is run,. It is fairly simple to implement in code due to fewer steps as opposed to something like the „Prim's Maze" algorithm, and is more efficient than that of „Aldus/Broder".

To set up this algorithm, you must start with a perfect maze, regardless of how simple it is, and as such a very basic arrangement like in Fig. 1 can be used.

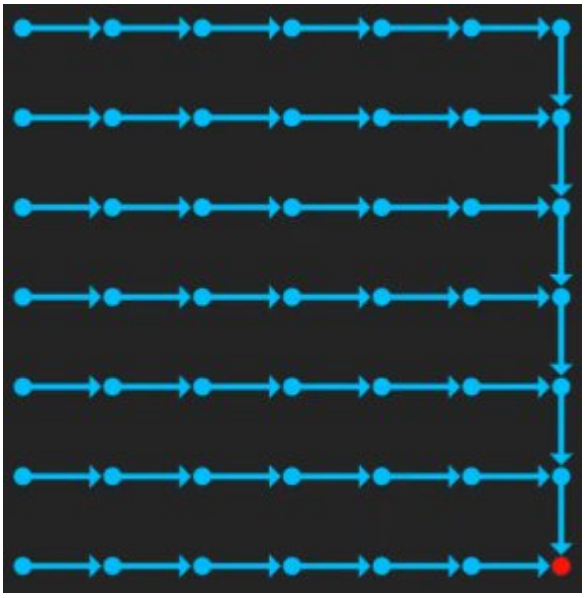

Figure 1 – Diagram of rudimentary map layout

From this point on, rather than conceptualising a maze as a series of walkways separated by walls, it's more practical to see them as a series of nodes with connections that draw the possible paths. In origin shift, each node has a direction that then dictates what node it's connected to. If we map this to the previous arrangement, then every row is pointing left, and the final column points down – leading to an array of notes that all point to a root node, referred to as the „Origin". Origin shift is based around manipulating the position of the origin over the course of three recurring steps.

The first step is to choose a random direction from the origin, ensuring it points within the bounds of the map. The node in that direction becomes the new origin, and the new origin gets stripped of its direction. These steps are iterated over and over, as the origin performs a random walk across the map, breaking down pathways that all end up leading to it somehow, since any node has either one or no direction, it is impossible for a loop to occur.

The initial concept for this game was to give the player 3 lives and a set amount of time per life, to go around the maze to reach the origin, with the map shifting every time the player reached that goal, and the player being awarded points for it, and the map shifting if they ran out of time, reducing how many lives the player had left. However during the programming phase of the project, I shifted to a concept more centred around having one life, with a timer that constantly counts down and gets partially replenished when the origin is reached. This concept seemed significantly more fun, with its constantly reducing timer providing a sense of tension and urgency with gameplay, all the while making it less clunky to play.

Experimental Method

The first task that I had to undertake was to lay out a grid of nodes to work with. Using the Raylib C library, I opted for a game window that was built relative to the size of the map, for the sake of simplicity. An initial arbitrary size of 20 by 20 cells, with each cell being 40 pixels wide and tall. Setting everything out with equal heights and widths made things much easier to work with.

From this grid of nodes I initially opted to represent them as circles, with different colours representing what direction they had (fig. 2 & fig 3) – as i felt that would be simpler and quicker to implement to debug whether the algorithm was working correctly or not – with better visual representation later. (The white square in fig.3 was the displaced origin)

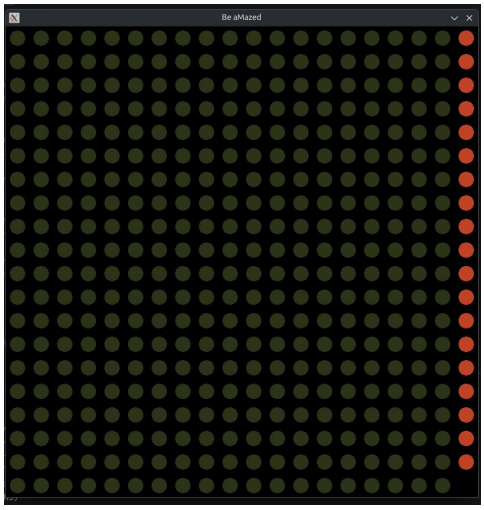

Figure 2 - Initial node mapping

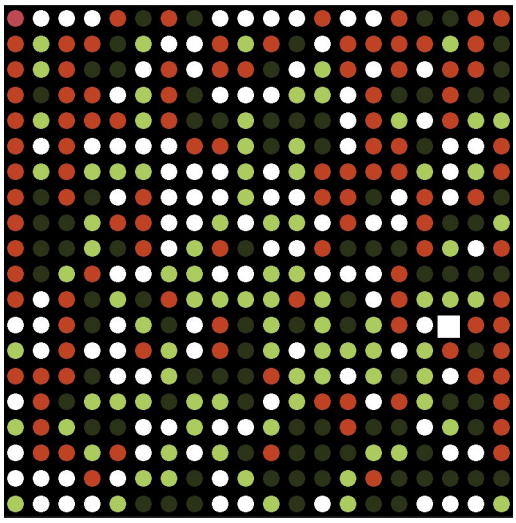

Figure 3 - Shuffled map, represented by coloured nodes

Once the algorithm functioned correctly (this took some time, as keeping the origin from wandering off the screen seemed to be a bit more of a task than initially anticipated), I made the representation of the maze more readable, by imposing white rectangles onto a black background (fig 4) and slightly elongating or shortening them in the respective axis that they were pointing.

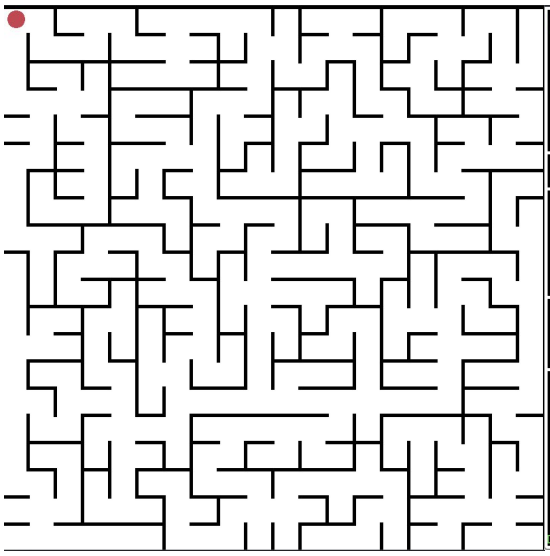

Figure 4 - Map visualisation

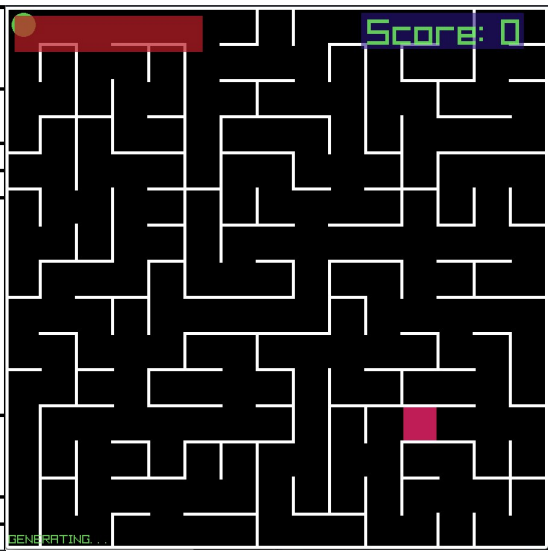

Figure 5 - Finalsied game appearance

This was a simple and effective method of displaying the actual maze mapping, and the only changes that were made to that implementation were that I decided to reverse the colours (to black squares on a white background) as I liked the „Pong" era aesthetic, as well as adjusting the sizes of the rectangles on the borders to make borders around the edges of the map more even (fig. 5).

Enabling player movement was also quite a challenge, as I had to figure out the logic that it would take to prevent the player from moving through walls. The creation of this feature was quite perplexing, as my initial implementation of player movement was based on a different co-ordinate system from the one used to draw the map. This resulted in the player being a growing power of 2 grid squares away from where the actual „hitbox" was being registered.

After realising that this was the issue, however, the player drawing mechanics were thrown completely off balance – as my method to draw the player was much more rudimentary. In the end – I determined that this was the equation that was needed to draw the player square accurately on all axes to maintain balance with the hitbox.

$$x = \left(C\frac{2P+1}{2}\right) + \frac{C}{2}$$

Where x is the respective x (or y, this equation is used in both axes) co-ordinate (in pixels), C is the cell size (in pixels), and P is the grid position of the player. The C/2 is a half-cell offset, neceessary since the circle is drawn from the centre of the cell, whereas the rest of the maths was determined by dividing the desired pixel position by the cell size for multiple results (20 px, 60 px, 100 px, etc.) and finding the correlating factor. This equation allows for use regardless of the size of the cells, nor their quantity.

The wall checking logic was quite fun to develop as well. The fact that each cell had a direction made for quite a straightforward set of rules to apply for each movement axis. To be able

to implement this I also had to introduce a new value to the player model – the canMove[direction] boolean that would be constantly off unless logic permitted it to. The hitbox logic functions as fig.6 and fig. 7 below represent (these examples are specifically for movement in the right direction; there are adjustments to the variables used in different directions).

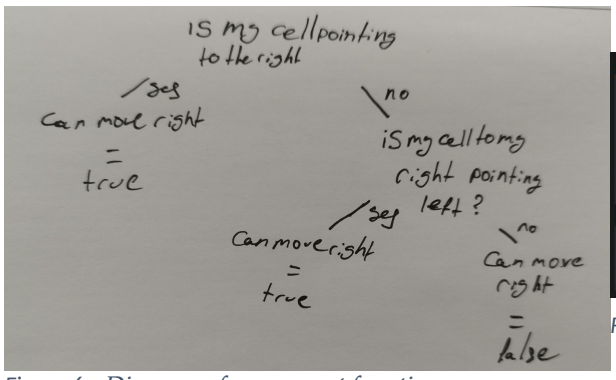

Figure 6 - Diagram of movement function

```
if   ((maze->visibleMap[ (int)player->position.x]      [(int) player->position.y].direction.x ==  1)  ||
     (((maze->visibleMap[((int)player->position.x)]     [(int) player->position.y].direction.y != 0)   ||
      (maze->visibleMap[((int)player->position.x)]     [(int) player->position.y].direction.x == -1)) &&
      (maze->visibleMap[((int)player->position.x + 1)] [(int) player->position.y].direction.x == -1)))
     {
     player->canMoveRight = true;
     }
else
     {
     player->canMoveRight = false;
     }
```
Figure 7 - Code for movement function

The initial design of this program was intended to have a fixed blocking loop of roughly cell-count cubed iterations (with 20 cells that would be 8000 iterations of the origin shift algorithm, I found cell-count cubed was the most reliable amount of calculations to fully shuffle a maze) every time the player reached the origin. This proved to work well as a simple implementation, however when I took the code from my PC and tried it on my laptop, the program would hang every time the loop was called. I believe this was something to do with my laptop processor being weaker and the calculation taking long enough for my operating system to consider the process as „not responding" due to the lack of updates in the program.

To work around this, I decided to try and design a non blocking generation function. I achieved this by having a map switching function (fig. 8). At all times there are two maps, the visible map, that is used to read hitbox positions and the visible origin position, and an invisible one that is being generated constantly in the background. This allows for a function that constantly runs in parallel to the normal operation of the game and does not block it. To indicate background generation I made a function that displated text on screen, visible to the player. Due to the great amount of calculations that are needed to be performed, I increased the frame rate at which the game ran to 1000 frames per second, up from 60. Since 8000~ calculations were expected that would save the amount of time needed to calculate the new map. This number wasn't raised to 8000+ because I did not want to overload weaker processors, but with the 1000 FPS it seemed to work on every machine I tested on (I tested on around 5 machines of varying CPU, operating system and hardware).

```
void PrepNextMap(struct Maze* maze) {
    if (maze->iterations<CELL_COUNT*CELL_COUNT*CELL_COUNT) {
        Shift(maze);
        maze->iterations++;
        maze->isGenerating = true;
    }
    else {
        maze->isGenerating = false;
    }
}
```
Figure 8 - Code for background generation of maze

Finally, I needed to implement a timer function and score, as well as a death mechanic. The score was easy to implement, as I just needed to add another variable and add to it every time „CheckOnOrigin" returns true. I initially wanted the timer to be numerical, with fractions of a second counting down – but the time libraries seemed to be clunky to use, and I wanted something easier to implement. Thus, I opted to simply have a rectangle of some sort of width, that would reduce in width as the game ran (as featured in Fig. 5). This ended up being extremely simple and a much more engaging visual representation of the remaining time, seeing the bar refill and drain with every map refresh ended up being very exciting.

Results

This was a very exciting and fun project to do – I feel like I learned a lot about C and C++ (I made a lot of the early code in the wrong language by mistake, which later required translation). I shared my game with a few friends, and a lot of them found the game fun, but quite difficult – so I reduced the size of the map.

There is a bug that ended up being part of the final game, but I reckoned that it was insignificant enough to ignore – as it would take more time to fix the issue than it would be worth. This bug involved the map switching function. When the maps switched before the generation went once over fully (or at least if the origin hadn't gone back over itself at least once), then a white square would appear. Technically it does not affect gameplay, as that square does not prevent the maze from being perfect, and it is impossible to enter the square and 'get stuck'. Due to this, I left the bug in due to the tradeoff of time saved.

Another few improvements I would have made would be HUD-related. I found a lot of the time that when the route necessary went under HUD elements, it was difficult to discern whether or not there was a wall in place of the HUD. A function to shift the HUD to the bottom of the screen whenever the player was underneath HUD elements was attempted, but it ended up running over my deadline for when to complete the game. Another similar function would be to simply reduce the opacity of those elements whenever the player was underneath, could be another way to improve such issues.

With my mistake of writing the code in the wrong language, I will attribute that my friend did help me in the translation from C++ to C, however all of the fundamental game logic remained entirely unchanged, and all of the work remained my own. My ability to code in C was not the reason for help in translation, purely my ability to intepret something in an object oriented language into a language that isn't. All of the code was fundamentally written by me.

Conclusions

In conclusion, I found this to be a very fun project and a great way to get my mind back into programming after a while of being away from the subject. A thing that this project helped me realise was the difficulty of accurately balancing games. Balancing map size, timer amount, and amount of time granted per map completion were all things that required a careful balance that I remain unsure as to whether or not I achieved. The importance of avoiding magic numbers was also quite important – but I believe that I could have put more effort in with that, and it's something I will actively work on in future projects.